Fase 1 — Black–Scholes, movimiento browniano geométrico y convergencia de Monte Carlo

Código: `mcpricer/black_scholes.py`, `mcpricer/monte_carlo.py`, `experiments/phase1_convergence.py`

---

1. ¿Qué es una opción y qué estamos valorando?

Una **call europea** con strike `K` y vencimiento `T` te da el *derecho* (no la obligación) de comprar el activo a precio `K` en la fecha `T`.

- Si en `T` el activo vale `S_T > K`, ejerces: compras a `K` y vale `S_T` → ganas `S_T − K`.
- Si `S_T ≤ K`, no ejerces → ganas 0.

Así que el **payoff** (lo que cobras en `T`) es `max(S_T − K, 0)`. Una **put** es el derecho a *vender* a `K`: payoff `max(K − S_T, 0)`.

El problema: hoy no sabemos cuánto valdrá `S_T`. **¿Cuánto debería costar hoy ese contrato?**

2. La idea clave: valoración neutral al riesgo

Ingenuamente dirías "precio = valor esperado del payoff". Casi, pero con dos matices:

1. **Descontar**: 1 € dentro de un año vale menos que 1 € hoy. Con interés continuo `r`, se multiplica por `e^{-rT}`.
2. **¿Esperanza bajo qué probabilidades?** No las "reales". El argumento de Black–Scholes es que puedes **replicar** la opción con una cartera de activo + dinero en el banco que se reajusta continuamente. Si dos cosas pagan lo mismo en todos los escenarios, deben costar lo mismo hoy (si no, habría **arbitraje**: ganancia sin riesgo). Ese precio de replicación resulta **no depender** de cuánto crees que va a subir el activo.

La consecuencia matemática es que el precio se puede calcular *como si* el activo creciera en media al tipo libre de riesgo `r`. A ese mundo ficticio se le llama **medida neutral al riesgo** `Q`:

$$V_0 = e^{-rT}\,\mathbb{E}^{Q}\big[\text{payoff}(S_T)\big]$$

Esta fórmula es *todo* Monte Carlo: **simula muchos `S_T` en el mundo Q, calcula el payoff de cada uno, haz la media y descuenta.**

> Ojo a la interpretación: Q no es "lo que creemos que pasará". Es una herramienta de cálculo que da el precio sin arbitraje. Por eso en el código la deriva es `r` y no un rendimiento esperado `μ`.

3. El modelo del activo: movimiento browniano geométrico (GBM)

Black–Scholes supone que el precio sigue:

$$dS_t = r\,S_t\,dt + \sigma\,S_t\,dW_t$$

Léelo así: en un instante pequeño `dt`, el **rendimiento** `dS/S` es una parte determinista `r·dt` más un ruido `σ·dW`, donde `dW` es normal con media 0 y varianza `dt` (eso es el **movimiento browniano** `W`). `σ` es la **volatilidad**: cuánto ruido hay.

Que sea "geométrico" significa que el ruido es *proporcional* al precio → el precio nunca es negativo, y los rendimientos (no los precios) son lo que es normal.

Solución exacta

Esta ecuación tiene solución cerrada:

$$S_T = S_0 \exp\Big(\big(r - \tfrac{1}{2}\sigma^2\big)T + \sigma\sqrt{T}\,Z\Big), \qquad Z \sim N(0,1)$$

Por eso en `simulate_terminal` basta con generar `n` normales `Z` y aplicar esta fórmula: **no hace falta simular el camino paso a paso** para una opción europea, porque el payoff solo depende de `S_T`. (Cuando lleguemos a asiáticas y barreras, que dependen del camino entero, sí lo haremos.)

¿Por qué aparece el `−σ²/2`? (la parte que más confunde)

Queremos que en el mundo Q el activo crezca en media a `r`: `E[S_T] = S_0 e^{rT}`. Un hecho de la normal: si `X ~ N(m, v)`, entonces

$$\mathbb{E}[e^X] = e^{m+v/2}$$

(la exponencial es convexa, así que los escenarios buenos pesan más que los malos — **desigualdad de Jensen**). Aquí `X = (r − σ²/2)T + σ√T Z`, con `m = (r − σ²/2)T` y `v = σ²T`, así que

$$\mathbb{E}[S_T] = S_0\, e^{(r-\sigma^2/2)T + \sigma^2 T/2} = S_0\, e^{rT} \checkmark$$

El `−σ²/2` está ahí exactamente para cancelar el `+σ²/2` que añade la convexidad. En cálculo estocástico esto sale del **lema de Itô** (la "regla de la cadena" para procesos aleatorios, que tiene un término extra de segundo orden). No necesitas Itô para entender el código: basta con el hecho de arriba sobre `E[e^X]`. Puedes comprobarlo tú: `simulate_terminal(...).mean()` con muchos `n` da ≈ `100·e^{0.05} = 105.13`.

4. La fórmula de Black–Scholes (nuestra referencia)

Calculando esa esperanza a mano (es una integral contra la densidad normal) sale:

$$C = S_0\,N(d_1) - Ke^{-rT}N(d_2), \qquad P = Ke^{-rT}N(-d_2) - S_0\,N(-d_1)$$

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}$$

con `N(·)` la función de distribución de la normal estándar (`norm.cdf`).

Intuición: `N(d_2)` es la probabilidad (en Q) de que la call acabe ejerciéndose, `S_T > K`. Así que `K e^{-rT} N(d_2)` es el valor actual de "pagar K si ejerces". El término `S_0 N(d_1)` es el valor actual de "recibir el activo si ejerces" (usa `d_1` y no `d_2` porque cuando ejerces el activo suele valer más de lo normal — está condicionado a escenarios altos).

Paridad put–call: comprar una call y vender una put con el mismo `K` paga `S_T − K` siempre, así que `C − P = S_0 − K e^{-rT}`. Lo comprobamos en `test_put_call_parity`. Es una forma barata de pillar errores en las

fórmulas.

Con `S_0=100, K=100, T=1, r=5%, σ=20%`: **call = 10.4506, put = 5.5735**.

5. El estimador Monte Carlo

Con `N` simulaciones independientes `Y_i = e^{-rT}·payoff(S_T^{(i)})`:

$$\hat{V}_N = \frac{1}{N}\sum_{i=1}^{N} Y_i$$

- **Ley de los grandes números**: `\hat V_N → V_0` cuando `N → ∞`. El estimador es **insesgado**: su media es exactamente el precio, para cualquier `N`.
- **Teorema central del límite (TCL)**: para `N` grande, `\hat V_N` es aproximadamente normal con media `V_0` y desviación típica

$$\mathrm{SE} = \frac{\sigma_Y}{\sqrt{N}}$$

donde `σ_Y` es la desviación típica de **un** payoff descontado (no confundir con la `σ` del activo). No conocemos `σ_Y`, pero la estimamos con la desviación típica muestral: `discounted.std(ddof=1)` (`ddof=1` divide por `N−1`, la versión insesgada de la varianza).

Por qué el error baja como 1/√N

La varianza de una media de `N` cosas independientes es `Var(Y)/N`. La desviación típica es su raíz: `σ_Y/√N`. Consecuencias prácticas:

- **Para dividir el error entre 10 necesitas 100 veces más simulaciones.** Eso es caro, y es la razón de existir de la **fase 2** (reducción de varianza): en vez de aumentar `N`, reducimos `σ_Y`.
- La tasa 1/√N **no depende de la dimensión** del problema. Una opción sobre 50 activos converge igual de rápido. Por eso Monte Carlo gana a otros métodos numéricos (árboles, EDPs) cuando hay muchas variables.

Intervalo de confianza al 95%

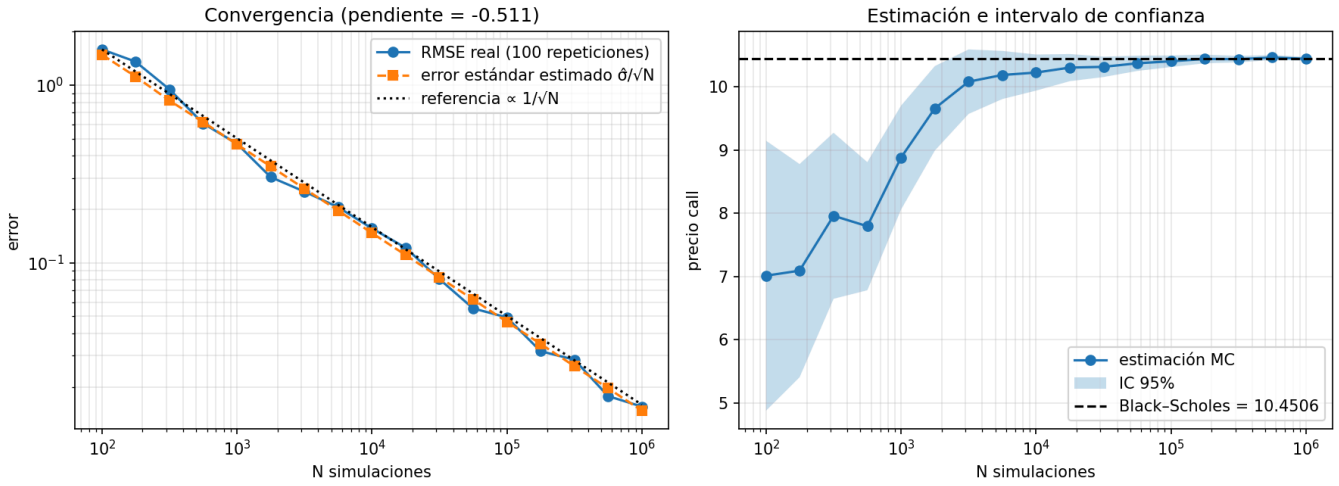
Como `\hat V_N` es ≈ normal, el 95% de las veces cae a menos de 1.96 errores estándar del valor real:

$$\left[\hat{V}_N - 1.96\,\mathrm{SE},\ \hat{V}_N + 1.96\,\mathrm{SE}\right]$$

Interpretación correcta: si repites el experimento muchas veces, el 95% de los intervalos que construyas contienen el precio verdadero. **Un** intervalo concreto lo contiene o no.

6. Resultados

```
python -m experiments.phase1_convergence
```



Panel izquierdo (log-log). En escala logarítmica en ambos ejes, `error = c/√N` se convierte en `log(error) = log(c) − ½·log(N)`: una **recta de pendiente −½**. La recta ajustada da **−0.511**. Dos curvas casi superpuestas:

- *RMSE real*: repetimos el cálculo 100 veces con semillas distintas para cada `N` y medimos el error cuadrático medio contra Black–Scholes. (Un solo error es muy ruidoso; promediar muestra la tendencia.)
- *Error estándar estimado*: lo que el propio Monte Carlo dice que es su error, sin conocer la respuesta. Que coincidan con el RMSE significa que **el SE es fiable** — y eso importa, porque en problemas reales (fase 4: asiáticas aritméticas, americanas) no tendremos fórmula para comparar y el SE será lo único que tengamos.

Panel derecho. Una sola semilla, aumentando `N`. Con `numpy`, los primeros `N` números de una semilla son los mismos, así que esta curva es la media acumulada de una sola tirada larga. Al principio salta mucho; el intervalo azul se estrecha como 1/√N.

Fíjate en que con `N = 100–600` el intervalo **no contiene** el valor real. No es un bug: (1) el 5% de las veces debe fallar, y como estos puntos comparten números aleatorios, fallan juntos; (2) con `N` pequeño el TCL aún no funciona bien — el payoff de una call es muy asimétrico (muchos ceros y unos pocos valores grandes), así que la media tarda en parecer normal.

Cobertura. En las 1700 repeticiones (17 valores de `N` × 100), el intervalo del 95% contuvo el precio real el **94.4%** de las veces. Exactamente lo que dice la teoría.

Tabla de errores frente a Black–Scholes (N = 1 000 000)

Tipo	K	Black–Scholes	Monte Carlo	Error	Error estándar	|error|/SE
call	80	24.5888	24.5905	+0.0017	0.0192	0.09
call	90	16.6994	16.6615	−0.0379	0.0173	2.19
call	100	10.4506	10.4411	−0.0095	0.0147	0.65
call	110	6.0401	6.0221	−0.0180	0.0116	1.54
call	120	3.2475	3.2497	+0.0022	0.0087	0.26

Tipo	K	Black–Scholes	Monte Carlo	Error	Error estándar	|error|/SE
put	80	0.6872	0.6897	+0.0025	0.0027	0.93
put	90	2.3101	2.3211	+0.0110	0.0054	2.06
put	100	5.5735	5.5802	+0.0067	0.0087	0.77
put	110	10.6753	10.6924	+0.0171	0.0120	1.43
put	120	17.3950	17.3859	−0.0091	0.0148	0.61

Cómo leerla: la columna importante es **|error|/SE**. Si Monte Carlo funciona, debería ser como el valor absoluto de una normal estándar: casi siempre < 2, raramente > 3. Aquí todo está bajo 2.2. Los dos casos de `K = 90` superan 2 *a la vez* porque la call y la put usan la misma semilla (`seed=K`), así que comparten escenarios: si esos escenarios salieron algo bajos, la call sale baja y la put alta. Es un buen ejemplo de **errores correlacionados** — y precisamente esa idea (usar los mismos números aleatorios a propósito) es la base de las griegas por diferencias finitas en la fase 3.

Fíjate también en que el **SE absoluto** de las puts fuera de dinero (`K=80`) es mucho menor (0.0027): casi todos los payoffs son 0, así que hay poca varianza. Pero en *relativo* (0.0027/0.69 ≈ 0.4%) es peor que la call at-the-money (0.0147/10.45 ≈ 0.14%). Las opciones muy fuera de dinero son difíciles para Monte Carlo porque pocos escenarios aportan información — esto motiva técnicas como el *importance sampling*.

7. Comprueba que lo has entendido

1. Cambia `sigma` a `0.4` en el script. ¿Qué pasa con el precio de Black–Scholes y con el error estándar? ¿Por qué tiene sentido que suban los dos?
2. Quita el `-0.5 * sigma**2` en `simulate_terminal` y ejecuta los tests. ¿Cuáles fallan y por qué el precio sale demasiado alto?
3. Sin ejecutar nada: si con `N = 10 000` el SE es 0.15, ¿cuántas simulaciones necesitas para un SE de 0.005?
4. ¿Por qué no podemos usar la misma idea "una normal por escenario, sin pasos de tiempo" para una opción cuyo payoff depende del **máximo** del precio durante el año?

▶ Respuestas